

TASK 2 / ЗАВДАН

FILTERING DATA IN MYSQL: THE ORBITAL PROBE REGISTRY ФІЛЬТРАЦІЯ ДАНИХ У MYSQL: РЕЄСТР ОРБІТАЛЬНИХ ЗОНДІВ

WHERE, AND, OR, NOT, LIKE, IN, BETWEEN, IS NULL, IFNULL, COALESCE.

Environment: MySQL 8.0+ (MySQL Workbench or the mysql command-line client).

WHERE, AND, OR, NOT, LIKE, IN, BETWEEN, IS NULL, IFNULL, COALESCE.

Середовище: MySQL 8.0+ (MySQL Workbench або клієнт командного рядка mysql).

Scenario · Сценарій

The Orbital Administration operates a fleet of automatic probes that survey planets and moons of the outer system. Each record stores the probe code, its name, the surveyed body, the mission status, the power output of the reactor, the launch year, the orbit altitude and the date of the last received signal. Two of the columns are optional: the altitude is unknown for probes that have not reached a stable orbit, and the date of the last signal is absent for probes that have never answered a call.

Орбітальне управління керує флотом автоматичних зондів, які досліджують планети та супутники зовнішньої системи. Кожен запис містить код зонда, його назву, досліджуване тіло, стан місії, потужність реактора, рік запуску, висоту орбіти та дату останнього прийнятого сигналу. Два стовпці є необов'язковими: висота невідома для зондів, які не вийшли на стабільну орбіту, а дата останнього сигналу відсутня для зондів, що жодного разу не відповіли на виклик.

0. Environment setup · Підготовка середовища

The statements below are outside the current topics and are given ready to copy and paste. Execute the whole block once. CREATE DATABASE creates the storage container, USE selects it as the default schema for the session, CREATE TABLE defines the structure, and INSERT loads twelve records. The columns orbit_km and last_contact are declared NULL, so a record may be stored without a value in them; the keyword NULL in the INSERT list marks exactly such a case.

Наведені нижче оператори виходять за межі поточних тем і подані у готовому до копіювання вигляді. Виконайте блок один раз. CREATE DATABASE створює контейнер зберігання, USE обирає його як стандартну схему сеансу, CREATE TABLE визначає структуру, а INSERT завантажує дванадцять записів. Стовпці orbit_km і last_contact оголошено як NULL, тому запис можна зберегти без значення в них; ключове слово NULL у переліку INSERT позначає саме такий випадок.

```
DROP DATABASE IF EXISTS orbital_survey;
CREATE DATABASE orbital_survey CHARACTER SET utf8mb4;
USE orbital_survey;

CREATE TABLE survey_probe (
  probe_id      INT AUTO_INCREMENT PRIMARY KEY,
  probe_code    VARCHAR(12) NOT NULL,
  probe_name    VARCHAR(40) NOT NULL,
  target_body   VARCHAR(20) NOT NULL,
  mission_status VARCHAR(15) NOT NULL,
  power_output  DECIMAL(5,1) NOT NULL,
  launch_year   YEAR NOT NULL,
  orbit_km      INT NULL,
  last_contact  DATE NULL
) ENGINE = InnoDB;

INSERT INTO survey_probe (probe_code, probe_name, target_body,
  mission_status, power_output, launch_year, orbit_km, last_contact)
VALUES ('PX-101-A', 'Aurora Scout', 'Mars', 'Active', 240.5,
```

```

2231, 420, '2247-03-14'),
('PX-102-B', 'Kraken Eye', 'Europa', 'Active', 180.0,
2233, 310, '2247-03-02'),
('SR-210-A', 'Pathfinder II', 'Titan', 'Standby', 95.5,
2236, NULL, '2246-11-27'),
('PX-103-C', 'Iron Sentinel', 'Mars', 'Lost', 310.0,
2228, 500, NULL),
('SR-211-B', 'Nova Probe', 'Ceres', 'Active', 145.0,
2240, 260, '2247-03-18'),
('TL-330-A', 'Silent Drift', 'Io', 'Lost', 88.0,
2229, NULL, NULL),
('PX-104-D', 'Helios Scout', 'Europa', 'Standby', 205.5,
2238, 350, '2247-01-09'),
('SR-212-C', 'Vega Relay', 'Titan', 'Active', 275.0,
2242, 610, '2247-03-21'),
('TL-331-B', 'Ghost Signal', 'Io', 'Retired', 60.0,
2225, NULL, '2244-07-30'),
('PX-105-E', 'Ember Sail', 'Ceres', 'Active', 190.5,
2244, 275, '2247-03-11'),
('SR-213-D', 'Titan Wake', 'Titan', 'Retired', 120.0,
2226, 480, '2243-05-16'),
('TL-332-C', 'Solar Anvil', 'Mars', 'Standby', 330.0,
2239, 540, '2247-02-25');

```

1. WHERE

WHERE is placed after FROM and keeps only the rows for which its condition is true. The condition is built from comparison operators: = for equality, <> for inequality, and >, >=, <, <= for ordered values. Character constants are enclosed in single quotes, numeric ones are written without quotes.

WHERE розміщується після FROM і залишає лише ті рядки, для яких його умова істинна. Умову будують з операторів порівняння: = для рівності, <> для нерівності та >, >=, <, <= для впорядкованих значень. Символьні константи беруть в одинарні лапки, числові записують без лапок.

```

-- 1.1 Probes that are still working / Зонди, які ще працюють
SELECT probe_code, probe_name, target_body
FROM survey_probe
WHERE mission_status = 'Active';

-- 1.2 High-power reactors / Реактори великої потужності
SELECT probe_name, power_output
FROM survey_probe
WHERE power_output > 200;

```

2. AND, OR, NOT

AND requires both conditions to hold, OR requires at least one, NOT reverses the result of the condition that follows it. The operators are evaluated in the order NOT, AND, OR, so an OR written next to an AND without parentheses is applied last and changes the meaning of the query. Parentheses set the order explicitly and are the reliable way to express a group of alternatives narrowed by a further requirement.

AND вимагає виконання обох умов, OR - принаймні однієї, NOT змінює результат умови, що стоїть після нього, на протилежний. Оператори обчислюються в порядку NOT, AND, OR, тому OR, записаний поряд з AND без дужок, застосовується останнім і змінює зміст запиту. Дужки задають порядок явно і є надійним способом виразити групу альтернатив, звужену додатковою вимогою.

```

-- 2.1 Working probes at Mars / Робочі зонди біля Марса
SELECT probe_name, power_output
FROM survey_probe
WHERE target_body = 'Mars' AND mission_status = 'Active';

-- 2.2 Two states, one power requirement / Два стани, одна вимога

```

```

SELECT probe_name, mission_status, power_output
FROM survey_probe
WHERE (mission_status = 'Active' OR mission_status = 'Standby')
      AND power_output >= 200;

-- 2.3 Everything except the lost probes / Усе, крім втрачених зондів
SELECT probe_name, mission_status
FROM survey_probe
WHERE NOT mission_status = 'Lost';

```

3. LIKE, IN, BETWEEN

LIKE compares a character value with a pattern in which the sign % stands for any sequence of characters, including an empty one, and the sign _ stands for exactly one character. IN checks membership in a list of values and replaces a chain of equalities joined by OR. BETWEEN sets a range and includes both bounds, so it is equivalent to a pair of conditions with >= and <=. Each of the three operators accepts the prefix NOT, which inverts the check.

LIKE порівнює символічне значення із шаблоном, у якому знак % позначає будь-яку послідовність символів, зокрема порожню, а знак _ - рівно один символ. IN перевіряє належність до переліку значень і замінює ланцюг рівностей, поєднаних через OR. BETWEEN задає діапазон і включає обидві межі, тому дорівнює парі умов з >= та <=. Кожен із трьох операторів приймає префікс NOT, який інвертує перевірку.

```

-- 3.1 Probes of the PX series / Зонди серії PX
SELECT probe_code, probe_name
FROM survey_probe
WHERE probe_code LIKE 'PX-%';

-- 3.2 Names that end with the word Scout / Назви, що закінчуються на Scout
SELECT probe_code, probe_name
FROM survey_probe
WHERE probe_name LIKE '%Scout';

-- 3.3 Three bodies of the outer system / Три тіла зовнішньої системи
SELECT probe_name, target_body
FROM survey_probe
WHERE target_body IN ('Europa', 'Titan', 'Ceres');

-- 3.4 Launches of a nine-year period / Запуски дев'ятирічного періоду
SELECT probe_name, launch_year
FROM survey_probe
WHERE launch_year BETWEEN 2230 AND 2238;

```

4. NULL

NULL denotes an absent value and is not a zero, not an empty string and not a space. A comparison of NULL with any value through = or <> yields neither true nor false but the third logical result, unknown, and a row with such a result is not included in the answer. Absence is therefore checked by the operators IS NULL and IS NOT NULL. A substitute for the missing value is produced by IFNULL with two arguments or by COALESCE, which returns the first of its arguments that is not NULL.

NULL позначає відсутнє значення і не є ані нулем, ані порожнім рядком, ані пробілом. Порівняння NULL з будь-яким значенням через = або <> дає не істину і не хибність, а третій логічний результат - невизначеність, і рядок з таким результатом до відповіді не потрапляє. Тому відсутність перевіряють операторами IS NULL та IS NOT NULL. Замінник пропущеного значення дає функція IFNULL з двома аргументами або COALESCE, яка повертає перший зі своїх аргументів, що не є NULL.

```

-- 4.1 Comparison with NULL gives nothing / Порівняння з NULL марне
SELECT probe_name FROM survey_probe WHERE orbit_km = NULL;

-- 4.2 Probes that never answered / Зонди, які жодного разу не відповіли

```

```

SELECT probe_name, target_body
FROM survey_probe
WHERE last_contact IS NULL;

-- 4.3 Probes with a known orbit / Зонди з відомою орбітою
SELECT probe_name, orbit_km
FROM survey_probe
WHERE orbit_km IS NOT NULL;

-- 4.4 Substitutes for the missing values / Замінники відсутніх значень
SELECT probe_name,
       IFNULL(orbit_km, 0) AS orbit_km,
       COALESCE(last_contact, '2200-01-01') AS last_contact
FROM survey_probe;

```

Control questions · Контрольні питання

1. Why does the condition `orbit_km = NULL` return no rows, and which operator has to be used instead?
2. How do parentheses change the result of a condition in which AND and OR are combined?
3. Which two wildcard characters are used by LIKE and how do they differ?

1. Чому умова `orbit_km = NULL` не повертає жодного рядка і який оператор слід використати замість неї?
2. Як дужки змінюють результат умови, у якій поєднано AND та OR?
3. Які два символи-замінники використовує LIKE і чим вони відрізняються?